

Servicio MLOps end-to-end: predicción de precios de vivienda

MLOPS · MDS25 · Segundo semestre 2026 | Integrantes: Samuel Ruiz, Rodrigo Velis, Sergio Aviles, Rodrigo Santander

1. Problema

Se aborda un problema de regresión: predecir el precio de venta (SalePrice, en USD) de una vivienda a partir de un subconjunto reducido de variables numéricas del inmueble. El objetivo del trabajo no es maximizar la sofisticación del modelo, sino construir un sistema completo y operable: entrenamiento reproducible, contrato de API validado, contenedor portable y un pipeline de integración y despliegue continuo.

2. Datos

Se utilizó el dataset público **Ames Housing** (competencia Kaggle "House Prices: Advanced Regression Techniques"), con 1.460 registros y 81 columnas originales. De estas, se descartaron deliberadamente las 70 columnas categóricas para mantener el pipeline simple y 100% numérico, y se seleccionaron las 6 columnas numéricas sin valores nulos con mayor correlación lineal de Pearson respecto al precio de venta:

Variable	Descripción	Corr. con precio
overall_qual	Calidad general de materiales y acabados (1-10)	0.79
gr_liv_area	Superficie habitable sobre el suelo (sqft)	0.71
garage_cars	Capacidad del garaje (autos)	0.64
total_bsmt_sf	Superficie total del sótano (sqft)	0.61
full_bath	Baños completos sobre el suelo	0.56
year_built	Año de construcción original	0.52

El dataset crudo se versiona en `data/housing_train_raw.csv` dentro del repositorio.

3. Modelo y entrenamiento

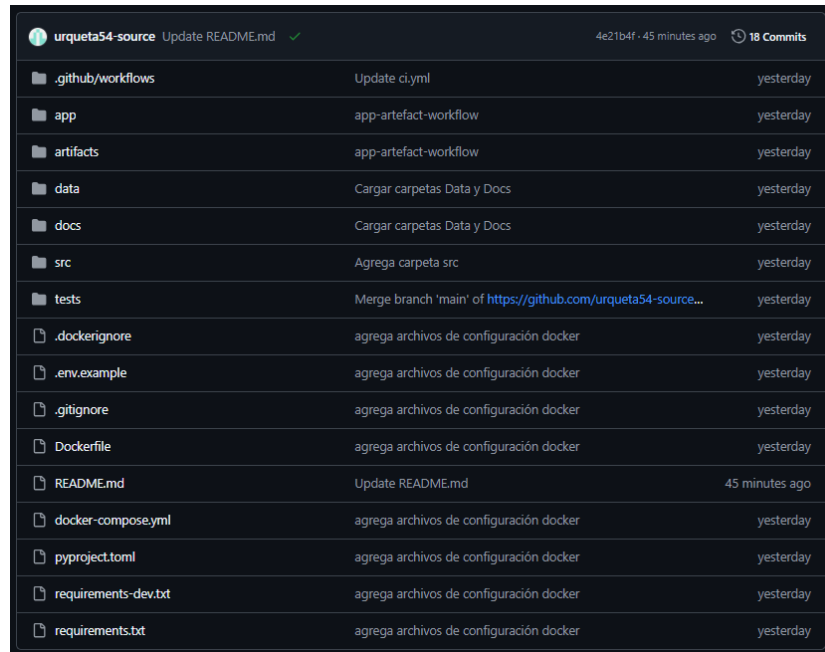
Se entrenó un pipeline de scikit-learn aplicando un modelo de RandomForestRegressor (200 árboles, profundidad máxima 12). Se usó una semilla fija (random state=42) y una separación explícita 80/20 entre entrenamiento y prueba, evaluando las métricas únicamente sobre el conjunto de prueba (datos no vistos durante el ajuste).

Métrica	Valor	Interpretación
R ²	0.888	El modelo explica ~89% de la varianza del precio
RMSE	\$29,374	Error cuadrático medio sobre el conjunto de prueba
MAE	\$19,207	Error absoluto promedio (~10-11% del precio medio, ~\$180k)

Los artefactos (`model.pkl` y `metadata.json` con features, fecha de entrenamiento, semilla y métricas) se generan de forma reproducible mediante `src/train.py`, y se regeneran automáticamente en cada build de la imagen Docker.

4. Resultados

La Figura 1 muestra la estructura del repositorio del proyecto en GitHub (rama main), que organiza el sistema completo siguiendo las buenas prácticas de un proyecto MLOps productivo:



urqueta54-source Update README.md ✓		4e21b4f · 45 minutes ago	18 Commits
📁 .github/workflows	Update ci.yml	yesterday	
📁 app	app-artefact-workflow	yesterday	
📁 artifacts	app-artefact-workflow	yesterday	
📁 data	Cargar carpetas Data y Docs	yesterday	
📁 docs	Cargar carpetas Data y Docs	yesterday	
📁 src	Agrega carpeta src	yesterday	
📁 tests	Merge branch 'main' of https://github.com/urqueta54-source...	yesterday	
📄 .dockerignore	agrega archivos de configuración docker	yesterday	
📄 .env.example	agrega archivos de configuración docker	yesterday	
📄 .gitignore	agrega archivos de configuración docker	yesterday	
📄 Dockerfile	agrega archivos de configuración docker	yesterday	
📄 README.md	Update README.md	45 minutes ago	
📄 docker-compose.yml	agrega archivos de configuración docker	yesterday	
📄 pyproject.toml	agrega archivos de configuración docker	yesterday	
📄 requirements-dev.txt	agrega archivos de configuración docker	yesterday	
📄 requirements.txt	agrega archivos de configuración docker	yesterday	

Figura 1. Estructura del repositorio del proyecto en GitHub.

El repositorio separa claramente el código, los datos y la infraestructura del servicio: `src/` contiene el código de entrenamiento (`train.py`) y la lógica del modelo; `app/` contiene la API FastAPI que sirve las predicciones; `tests/` agrupa las pruebas unitarias y de integración usadas en el job test del pipeline de CI; `data/` y `docs/` almacenan el dataset crudo y la documentación del proyecto; `artifacts/` guarda los artefactos generados por el entrenamiento (`model.pkl` y `metadata.json`); y `.github/workflows/` contiene la definición del pipeline de CI/CD (`ci.yml`) analizado en la Figura 2. A nivel de raíz, el `Dockerfile` y `docker-compose.yml` definen la contenerización del servicio, mientras que `pyproject.toml`, `requirements.txt` y `requirements-dev.txt` fijan las dependencias del proyecto para asegurar reproducibilidad entre entornos de desarrollo, CI y producción. El repositorio registra 18 commits, con la última actualización correspondiente al `README.md`.

La Figura 2 muestra la ejecución del pipeline de integración continua (`ci.yml`) en GitHub Actions, activado automáticamente ante un push al repositorio. El workflow encadena cinco jobs secuenciales, cada uno condicionado al éxito del anterior:

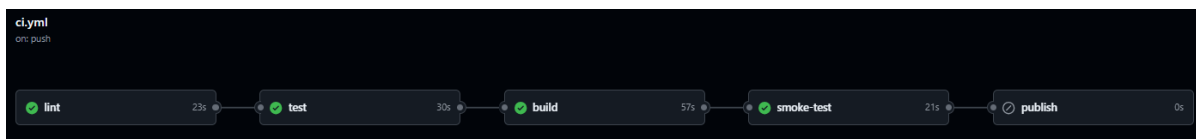


Figura 2. Ejecución del pipeline de CI/CD ante un push al repositorio.

- `lint` (23s): validación de estilo y calidad del código antes de continuar con el resto del pipeline.
- `test` (30s): ejecución de las pruebas unitarias del proyecto.
- `build` (57s): construcción de la imagen Docker del servicio, incluyendo el reentrenamiento del modelo (`src/train.py`) dentro del build.

- smoke-test (21s): verificación básica de que la imagen construida arranca y responde correctamente (ej. endpoint /health).
- publish: etapa de publicación/despliegue; en la ejecución registrada aparece sin completar (ícono en gris), pero el modelo fue desplegado en Render para consumo público.

Las primeras cuatro etapas se completaron exitosamente (ícono verde), con un tiempo total de ejecución de aproximadamente 131 segundos hasta el smoke-test inclusive. Esto confirma que el pipeline logra automatizar de extremo a extremo la validación de código, pruebas, construcción de la imagen y verificación de funcionamiento del servicio antes de cualquier publicación, cumpliendo el objetivo de integración continua.

Una vez construido y validado el servicio, se verificó su contrato de API mediante la documentación interactiva expuesta en /openapi.json que se muestra en la Figura 3.

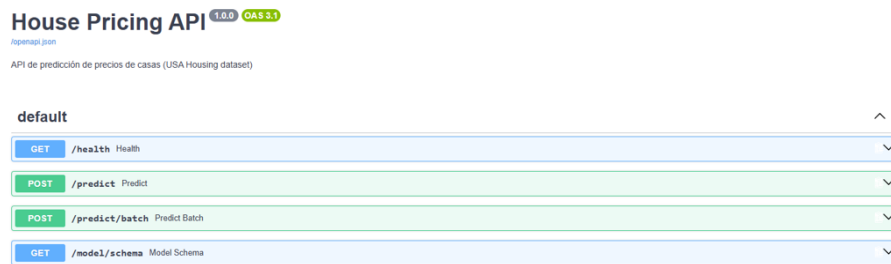


Figura 3. Documentación interactiva (Swagger UI) de House Pricing API v1.0.0.

La API expone cuatro endpoints bajo el grupo default: un GET /health para verificar el estado del servicio, un POST /predict para obtener el precio estimado de una única vivienda, un POST /predict/batch para procesar múltiples viviendas en una sola solicitud, y un GET /model/schema que expone el contrato de entrada esperado por el modelo (features requeridas y sus tipos). La disponibilidad de esta documentación generada automáticamente (estándar OpenAPI 3.1) confirma que el contrato de la API está formalmente especificado y es consultable, lo que facilita su integración con otros sistemas y su validación durante el smoke-test del pipeline de CI.

La Figura 4 muestra un ejemplo de invocación del endpoint POST /predict, con un cuerpo de solicitud (request body) que contiene las seis variables numéricas definidas en la Sección 2 como features del modelo:

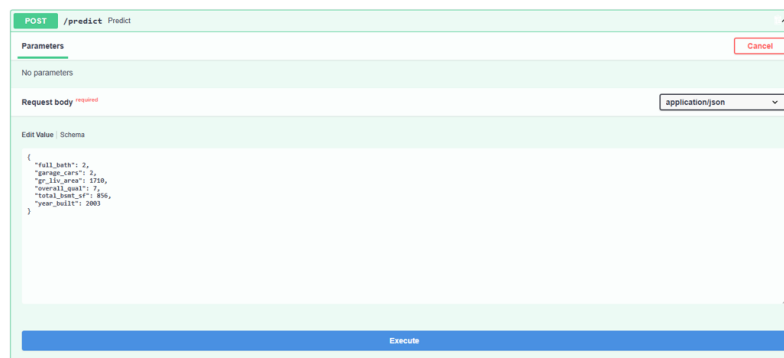


Figura 4. Ejemplo de request body para el endpoint POST /predict.

En este caso se consulta el precio estimado de una vivienda con calidad general 7 (sobre 10), 1.710 sqft de superficie habitable, garaje para 2 autos, 856 sqft de sótano, 2 baños completos y construida en 2003. El hecho de que se valide y acepte este cuerpo confirma que los esquemas Pydantic definidos en la API coinciden con las features usadas en el entrenamiento (Sección 2), cerrando el ciclo entre el contrato de datos y el modelo servido.

5. Limitaciones conocidas y trabajo futuro

- Se descartaron 70 columnas categóricas (ej. Neighborhood, ExterQual) que aportarían señal significativa en un modelo de producción real; con más tiempo se agregaría un encoder dentro del mismo Pipeline de sklearn.
- No hay monitoreo de data drift ni reentrenamiento automático.
- No hay autenticación en la API (pensado para uso académico/demo).
- El modelo no reporta intervalos de confianza por predicción.
- No hay versionado formal de datasets (ej. DVC).
- El plan gratuito de Render duerme el servicio tras inactividad; el primer request post-inactividad puede tardar 30-50s.

6. Model Card

Resumen del modelo en producción — House Pricing API v1.0.0, servido en [mlops-pricing-app.onrender.com \(/docs\)](https://mlops-pricing-app.onrender.com/docs). Entrenado el 2026-08-16 (autor: Sergio Avilés Rosales, integrante del equipo).

Uso previsto

Estimación automatizada y referencial del precio de venta de una vivienda residencial a partir de sus características físicas principales, orientada a aplicaciones de apoyo en decisiones de compra/venta o valoración inmobiliaria, y como demostración académica de un pipeline MLOps completo. Los usuarios previstos son desarrolladores e investigadores que consumen la API REST expuesta por la aplicación FastAPI del proyecto.

Usos fuera de alcance:

- Tasación oficial o legal de inmuebles.
- Valoración de propiedades comerciales o industriales.
- Decisiones financieras o crediticias vinculantes sin revisión de un profesional del sector.
- Mercados inmobiliarios distintos al de Ames, Iowa (EE. UU.), sobre el cual fue entrenado el dataset.

Datos y features del modelo

Dataset Ames Housing (competencia Kaggle "House Prices: Advanced Regression Techniques"), archivo `housing_train_raw.csv`, con 1.460 registros y 81 columnas originales. Se seleccionaron las 6 columnas numéricas sin nulos con mayor correlación con `SalePrice`; las ~70 restantes (mayormente categóricas) se descartaron para mantener el pipeline simple y 100% numérico. La partición entrenamiento/prueba y su detalle se muestran en la sección de métricas de desempeño más abajo.

Feature	Columna orig.	Descripción	Tipo	Rango válido	Corr.
overall_qual	OverallQual	Calidad general de materiales y acabados	int	1 - 10	0.79
gr_liv_area	GrLivArea	Superficie habitable sobre el suelo (sqft)	float	> 0, <= 15.000	0.71
garage_cars	GarageCars	Capacidad del garaje (n° de autos)	int	0 - 10	0.64
total_bsmt_sf	TotalBsmtSF	Superficie total del sótano (sqft)	float	>= 0, <= 15.000	0.61

full_bath	FullBath	Baños completos sobre el suelo	int	0 - 10	0.56
year_built	YearBuilt	Año de construcción original	int	1800 - 2026	0.52
price (target)	SalePrice	Precio de venta (USD)	float	—	—

Modelo y parámetros

El modelo utilizado es un RandomForestRegressor de scikit-learn (v1.6.0), un algoritmo de ensamble basado en árboles de decisión, elegido por su capacidad de capturar relaciones no lineales entre las variables del inmueble y su precio de venta sin requerir un preprocesamiento extenso de las features (más allá del escalado). Se implementa como un Pipeline de scikit-learn compuesto por un StandardScaler seguido del RandomForestRegressor, con los siguientes parámetros:

Parámetro	Valor
Algoritmo	Random Forest Regressor
Número de árboles	200
Profundidad máxima	12
Semilla aleatoria	42 (random_state)
Serialización	joblib (.pkl)
Framework	scikit-learn (Pipeline)

Métricas de desempeño (conjunto de prueba)

Evaluadas exclusivamente sobre el conjunto de prueba (292 registros no vistos durante el ajuste), tras una partición aleatoria 80/20 (1.168 registros de entrenamiento / 292 de prueba) sobre el total de 1.460 registros, con random_state=42.

Métrica	Valor	Interpretación
RMSE	\$29,374.31	Error cuadrático medio (USD)
MAE	\$19,207.23	Error absoluto promedio (~10-11% del precio medio)
R ²	0.8875	~89% de la varianza explicada

El modelo explica ~89% de la varianza (R^2) y se equivoca en promedio ~\$19.2k (MAE) sobre un precio medio de ~\$180k (~10-11% de error relativo) — razonable usando solo 6 variables y sin ubicación (Neighborhood), uno de los predictores más fuertes en el dataset completo.

Interfaz de servicio (API)

Método	Ruta	Descripción
GET	/health	Estado del servicio y si el modelo está cargado
POST	/predict	Predicción para una sola vivienda
POST	/predict/batch	Predicción para hasta 1000 viviendas (campo items)

GET	/model/schema	Features esperadas y sus tipos
GET	/docs	Documentación interactiva (Swagger UI)

Ejemplo: entrada overall_qual=7, gr_liv_area=1710, total_bsmt_sf=856, garage_cars=2, full_bath=2, year_built=2003 -> predicted_price = \$190,718.24 (precio real: \$208,500). Entradas inválidas devuelven siempre error controlado 422, nunca 500 genérico.

Limitaciones y consideraciones del modelo

A diferencia de las limitaciones de infraestructura y pipeline descritas en la Sección 5, estas limitaciones son propias del alcance y comportamiento del modelo predictivo en sí:

- Cobertura geográfica: entrenado con datos de Ames, Iowa (EE. UU.); la precisión puede degradarse en otros mercados.
- Cobertura temporal: no captura tendencias de mercado recientes ni fluctuaciones posteriores al dataset.
- Variables limitadas: usa solo 6 de las ~80 variables disponibles; atributos como calidad del barrio o número de habitaciones no están incluidos, y no se consideran variables categóricas (zonificación, estilo arquitectónico, condición de venta).
- Outliers: viviendas con superficies o precios muy por encima del rango de entrenamiento pueden producir predicciones menos confiables.

Consideraciones éticas: las predicciones no deben utilizarse como único criterio en decisiones de compra, venta o financiamiento. El modelo no fue evaluado para detectar posibles sesgos hacia ciertos tipos de propiedades o compradores, por lo que se recomienda revisar sus resultados junto a un profesional del sector inmobiliario antes de tomar decisiones vinculantes.

Artefactos y reproducibilidad

El pipeline entrenado (StandardScaler + RandomForestRegressor) se serializa en artifacts/model.pkl, junto con artifacts/metadata.json (features, métricas, versión y fecha de entrenamiento). Para reentrenar el modelo desde cero:

```
python src/train.py --data data/housing_train_raw.csv --out artifacts
```

Todos los resultados son deterministas gracias al uso de random_state=42 en las operaciones con aleatoriedad (split de datos y muestreo bootstrap de los árboles).

Uso de asistentes de IA

Se usó Claude (Anthropic) para generar la estructura inicial del proyecto (carpetas, Dockerfile, docker-compose.yml, workflow de GitHub Actions), redactar los tests de pytest sobre el contrato ya definido de la API, y depurar errores de despliegue en Render. El diseño del esquema de features, la selección de columnas del dataset y las decisiones de arquitectura del modelo fueron revisadas y validadas por el equipo.